Agentic Band Assistant

Chris McKie

CSCI 264: Artificial Intelligence

Contents

1 Overview & Domain **2**
1.1 Project Overview & Goals . . . 2
1.2 Selected Domain . . . 2

2 Agent Architecture **2**
2.1 High-Level Overview . . . 2
2.2 Planner . . . 4
2.3 Generator . . . 4
2.4 Validator . . . 5
2.5 Executor . . . 5
2.6 Formatter . . . 5
2.7 Database . . . 6

3 Multi-LLM Call Flow **6**
3.1 Planner . . . 7
3.2 Generator . . . 7
3.3 Formatter . . . 8

4 Tools & APIs **8**

5 Validation & Safety Checks **8**
5.1 Layer 1: SQL Safety . . . 9
5.2 Layer 2: Business Logic . . . 9
5.3 Layer 3: Data Integrity . . . 9

6 Results & Discussion **9**
6.1 Demo Results . . . 9
6.1.1 Single Insert . . . 10
6.1.2 Batch Insert . . . 10
6.1.3 Select By Difficulty . . . 11
6.1.4 Validation Check . . . 11
6.2 Limitations & Expansion . . . 11

1 Overview & Domain

1.1 Project Overview & Goals

Most people think that the job of a band director is to teach students how to play instruments and conduct the students as they perform at concerts. While this is certainly an important part of the job, many people don't realize that the job has many more administrative responsibilities that happen behind the scenes, completely unnoticed. In addition to teaching and conducting during class time, countless hours outside of teaching hours are spent:

- Tracking hundreds of instruments and consumable equipment, repairs, and checkouts to students
- Selecting performance music based on student skills and instrumentation
- Financial and logistic planning for performances

The administrative role of a band director is incredibly time consuming, and often more time is spent on this than actually teaching. In addition, most band directors do not have the technical knowledge to automate these tasks, so they are often done manually and tracked in spreadsheets, where redundancy and data integrity problems can arise.

The goal of this project is to create an agent that helps band directors manage their instrument inventory, music selection, and ensemble planning. The band director simply tells the agent what to do through natural language, and the agent handles the task, with no technical or programming knowledge needed [1].

1.2 Selected Domain

The agent will work within the relational database domain. The administrative tasks of a band director involve large amounts of data, such as instrument inventory (including repair and checkout history), student information, finances, and a music library. In addition, the directors need to track relational information across these data, such as a student's history of checking out and returning instruments, if a student has completed a fundraiser related to an upcoming band trip, and whether or not there are enough instruments in the inventory to play a selected piece of music. For this reason, a relational database is the ideal choice for the agent.

2 Agent Architecture

2.1 High-Level Overview

The agent processes each request through a five-stage pipeline:

1. **Planner:** Classifies the request intent and produces structured data from the request
2. **Generator:** Produces parameterized SQL from the structured data

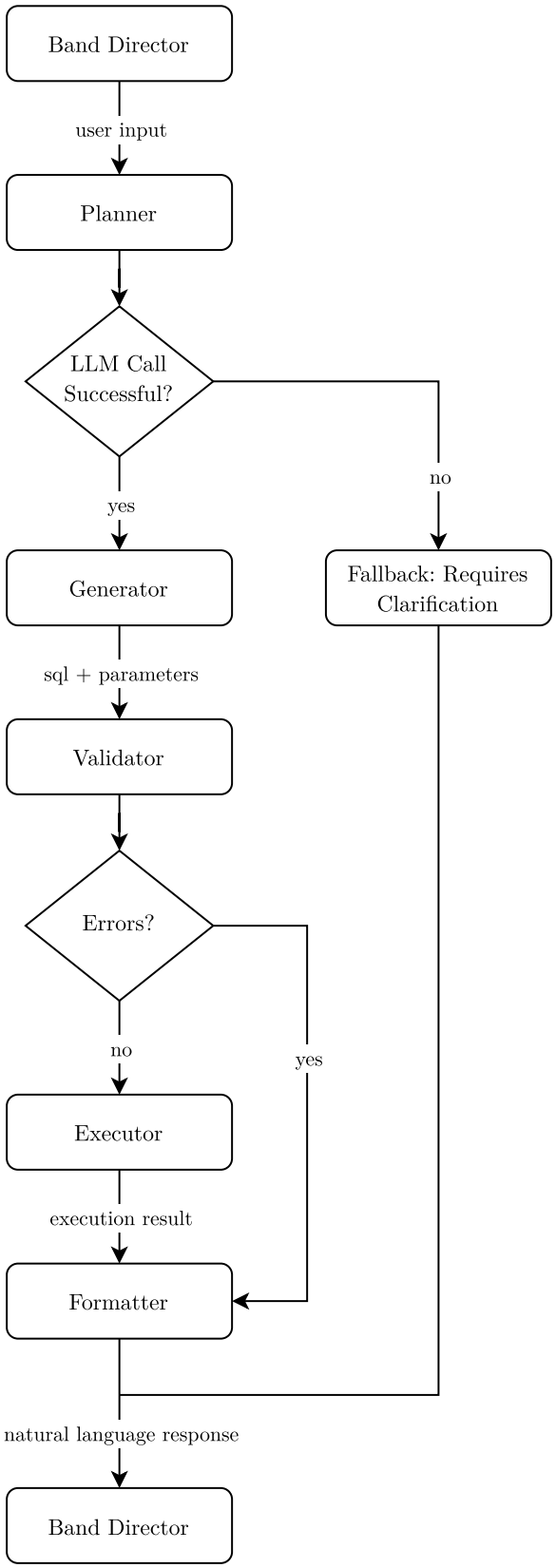

Figure 1: High-level diagram of the agent architecture.

3. **Validator:** Enforces safety and business rules on the SQL
4. **Executor:** Runs the SQL query in the database
5. **Formatter:** Turns the raw query results into a natural language response

Each stage has a defined input and output, making each component extendable and easy to test.

2.2 Planner

The Planner is the first step in the system. It receives the raw natural language prompt from the band director as a string, and uses an LLM call to organize this prompt into a structured plan describing what the user wants to do. The plan stores the following information inside a Python dictionary:

- **Intent:** The user's intended operation (INSERT, SELECT, UPDATE, or DELETE)
- **Entity:** The database entity being operated on
- **Is batch?:** Boolean value tracking whether this is a single operation or batch operation
- **Records:** Data records of a write operation (INSERT, UPDATE, and DELETE)
- **Filters:** Filter options for SELECT and UPDATE operations
- **Requires Clarification?:** True if the LLM cannot determine the intent
- **Clarification Question:** Follow up question for the user if the LLM requires clarification

The plan is the central component of the entire system, as it represents the user's request in a structured form that all downstream components read from in some way. By isolating the natural language prompt into the first layer, the rest of the system can operate on structured data while minimizing hallucinations from the LLM.

2.3 Generator

The Generator receives the plan and produces a parameterized SQL string with the values to bind it. For INSERT, UPDATE, and DELETE operations, the SQL generation is fully deterministic and is done using strictly Python. The Planner has already extracted column names and values, so the query can be built as a formatted string without using the LLM.

For SELECT operations, an LLM call is used to generate the queries. This is because queries may require complex joins, aliases, and special filter handling. If the LLM call cannot generate the SQL query, a safe full-table scan is used as a fallback.

All SQL produced by the Generator uses %s as placeholders, and the values are never directly inserted into the query. This allows the Validator and Executor to always receive parameterized queries regardless of what SQL operation is performed.

2.4 Validator

The Validator runs three layers of validation and safey checks against the SQL query and plan: The SQL Safety Layer, the Business Logic Layer, and the Data Integrity Layer. All three layers always run, any errors are collected into a list, and the errors list is evaluated at the end.

If errors are found, the entire operation is rejected and the error list is returned, where it is sent to the Formatter. The SQL query and parameters are sent to the Executor only if no errors are found. The Validator is discussed with more detail in Section 5.

2.5 Executor

The Executor receives the validated, parameterized SQL and executes the query. It is the only component that communicates directly with the Neon Postgres database. The query results are stored in a Python dictionary containing the following information:

- **Success?:** Boolean value indicating whether or not the query was successfully executed
- **Rows Affected**: Number of rows affected (Only used for INSERT, UPDATE, and DELETE operations)
- **Results:** A list of selected rows from the query (Only used for SELECT operations)
- **Error:** Any error message from the execution is stored here

All execution is done inside a try/except block. If an exception occurs, the connection is rolled back and the Executor sets the "Success" value to False and includes the error message. The result is then sent to Formatter to interpret the result and format it into a natural language response.

2.6 Formatter

The Formatter is the final stage of the system. It receives both the plan and results from previous layers, and returns a natural language English response for the band director.

If the execution is successful, the Formatter follows a system prompt based on the intent from the plan:

INSERT confirmations are kept at 1-2 sentences SELECT results are summarized with a cap of 25 rows UPDATE and DELETE responses include how many records were affected If the execution fails, it uses an error prompt instructing the LLM to explain what went wrong in natural language, with no SQL or database terminology.

If the system fails before a plan is created (for example, due to an API error), the Formatter bypasses the LLM call entirely and returns a hard-coded fallback string as an error message. This guarantees that the user always receives a response.

2.7 Database

While not a direct component in the agent pipeline, the database is a crucial part of the system, as this is where all the band data lives. The database contains five entity tables:

- **Students:** Students in the ensemble
- **Music:** Sheet music to be played
- **Instruments:** Instruments that a student can play or appear in music
- **Instrument Family:** Category that an instrument belongs to (brass, woodwind, string, percussion)
- **Instrument Inventory:** Inventory of instruments owned by the band program

In addition, there are three associative entity tables, matching relationships between the tables:

- **Plays:** Maps Students and Instruments (Which students play which instrument(s))
- **Arranged For:** Maps Music and the Instruments (The instruments arranged for the music piece)
- **Checkout History:** Maps Students and Instrument Inventory (Each instrument in the inventory keeps a history of each checkout, including the student, checkout date, and return date. A NULL return date indicates the instrument is currently checkout out to a student)

The schema of the database is the following:

students *(student_id, first_name, last_name, grade)*
music *(music_id, title, composer, difficulty)*
instruments *(instrument_name, family_id)*
instrument_family *(family_id, family_name)*
instrument_inventory *(serial_number, instrument_name, condition)*
plays *(student_id, instrument_name)*
arranged_for *(music_id, instrument_name, parts_needed)*
checkout_history *(checkout_id, student_id, serial_number, checkout_date, return_date)*

3 Multi-LLM Call Flow

The system tries to execute most of the operations using Python, minimizing the use of LLM calls as much as possible. This is for several reasons: First, using Python makes operations deterministic and easier to test. Relying on an LLM is inherently nondeterministic, and increases the chances of hallucinations, so LLM calls are only made when it the task cannot

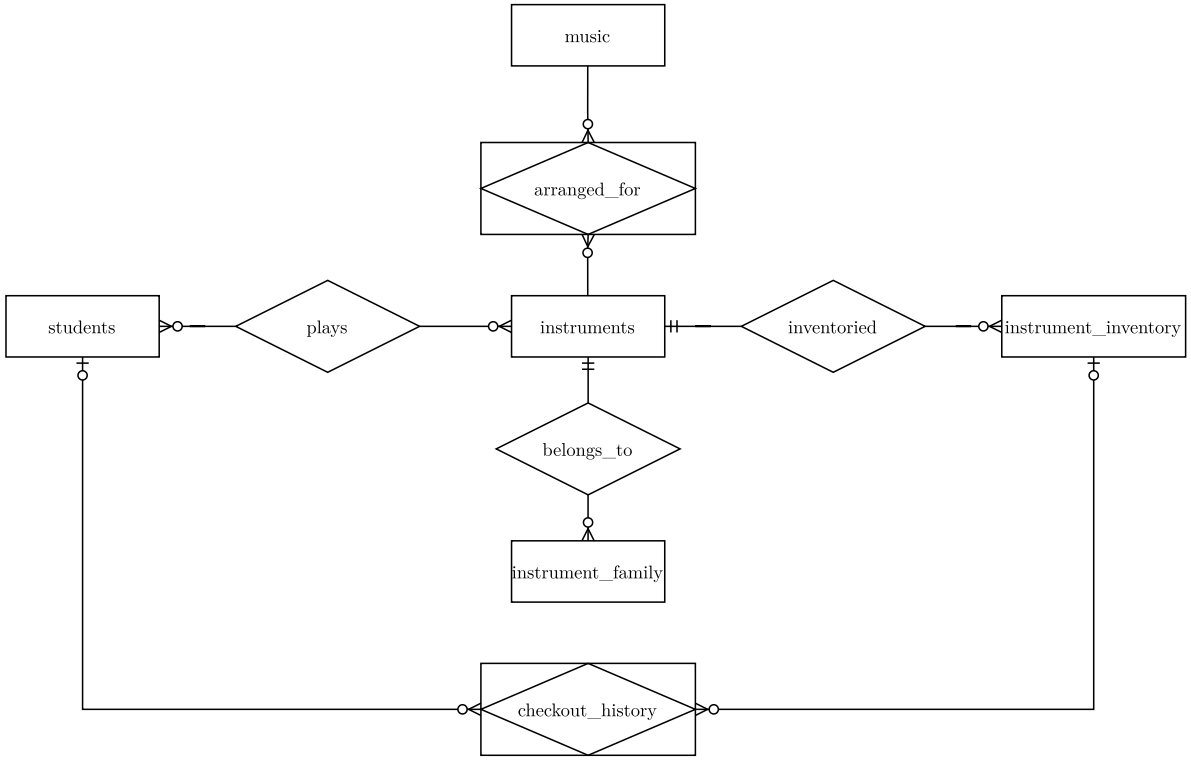

Figure 2: ER diagram of the database.

be reasonably done in Python alone. In addition, minimizing LLM calls also reduces the number of tokens needed to run the system.

For the LLM calls that are made, the system makes up to three separate LLM calls per request, each with a different role and system prompt. These calls are made in the Planner, the Generator, and the Formatter. By separating the LLM calls, each receives a well defined input and produces a well-defined output, reducing hallucinations and making each call independently testable

3.1 Planner

The first call handles all the natural language input from the user. The user's prompt is the input string, and the output is the plan, which is structured into a Python dictionary.

This is the only stage where unstructured or ambiguous data is processed. The structured dictionary acts as an intermediate representation of the user request for future LLM calls. Formatting it in this step allows all downstream stages (including future LLM calls) to operate on clean data rather than ambiguous text.

3.2 Generator

The second call is conditional, and *only* runs for SELECT queries. INSERT, UPDATE, and DELETE queries are generated deterministically in Python. Because the Planner already

extracted the column names and values, the query can be produced without the help of an LLM.

SELECT requires an LLM call because queries may span multiple tables with joins, aliases, and special filter logic that would be too difficult to capture in a query generating algorithm. The generator receives the plan (Python dictionary) as input and produces a parameterized SQL string and params list as output.

3.3 Formatter

The third call humanizes the query result by translating it into plain English. This call is crucial for the entire system, as this call allows technical database operations to be easily understood by band directors who do not understand how relational databases work.

The Formatter call receives the original user input as a string, the both the plan and execution result, both as structured dictionaries. One of five different system prompts depending on the intent and whether the operation succeeded or failed. If this call somehow fails or cannot be made (such as from an API error), a hardcoded fallback error message is returned instead.

4 Tools & APIs

This system uses Python to interact with several external tools and APIs. Specifically, the system connects to the Claude API for its LLM calls, and Neon Postgres for the database. In addition, the Python adapter psycopg2 is used to connect the Python code to the database. API keys for both Claude and Neon Postgres are stored in a .env file, securely keeping them out of the source code.

Tool / API	**Purpose**
Claude API	LLM calls for Planner, Generator (SELECT queries only), and Formatter
Neon Postgres	Postgres SQL database for storing band inventory data
psycopg2	Python adapter for connecting Python to the database
python-dotenv	Loading credentials and API keys from a separate .env file

Table 1: Tools & APIs used.

5 Validation & Safety Checks

The system runs validation and safety checks through the Validator component. The SQL queries and parameters go through the Validator before being run in the databagase, and queries will not be sent to the database unless they can pass through all three validation layers without any errors. This prevents any vulnerabilities through SQL injection, and also prevents actions that are outside the scope of the band agent, such as deleting tables or modifying data that should not be modified.

All validation is done through Python rules, which keeps validation and safety deterministic. Each of the three layers makes specific security checks.

5.1 Layer 1: SQL Safety

The SQL safety layer evaluates the SQL statement itself, making sure the query isn't malicious or harmful to the system, either intentionally or unintentionally. Tables in the query must be on a predetermined whitelist that the user will not see. In addition, any operation performed on a whitelisted table must be permitted. Specific keywords such as DROP, TRUNCATE, and ALTER are considered dangerous and are not allowed in operations. Finally, the number of placeholders in the string must match the number of provided parameters. Any query that fails these tests will trigger an error.

5.2 Layer 2: Business Logic

After validating the SQL safety, the second layer checks the business logic. These are rules that, while legal in SQL, do not follow predetermined rules related to band inventory systems. For example, the system is designed for high school band programs, so even though the grade value can technically be any integer, business rules require that it is between 9 and 12. Other business rules include a required music difficulty range between 1 and 6 as well as instrument conditions to be one of the specified values. SQL queries that do not follow the business rules will trigger an error.

5.3 Layer 3: Data Integrity

The third layer is the data integrity layer, which ensures the accuracy and reliability of the data stored in the database. For example, this prevents students with checked out instruments from being deleted. If a student has an instrument checked out, then deleting the student would erase the record of that instrument being checked out, which could cause the instrument to be lost from the inventory. A rule like this prevents critical data from being lost due to unintentional oversights in the requests. Other data integrity rules include preventing duplicate records in a batch request or a missing instrument name.

6 Results & Discussion

6.1 Demo Results

The demo for the system tests the agent functionality end-to-end using four basic operations: A single record insert, a batch record insert, a filter on the music library based on difficulty level, and invalid operation prevented by the validation layer. The demo shows a correct implementation of the data representations by logging each component's actions before printing the LLM output. However, the demo did expose some limitations and failures that required adjustments of the source code to work as expected.

```
response = chat("Add Emma Rodriguez, a 10th grader, to the student list.")
print(response)

[Planner] received: 'Add Emma Rodriguez, a 10th grader, to the student list.'
[Planner] plan: {'intent': 'INSERT', 'entity': 'students', 'is_batch': False, 'records': [{'first_name': 'Emma', 'last_name': 'Rodriguez',
[Generator] received plan: intent=INSERT, entity=students, is_batch=False
[Validator] is_valid=True, errors=[]
[Executor] received SQL: 'INSERT INTO students (first_name, last_name, grade) VALUES (%s, %s, %s)', is_batch=False
[Executor] success: rows affected: 1
Emma Rodriguez has been added to your student list as a 10th grader. You're all set!
```

Figure 3: Adding a student to the database.

```
response = chat(
    "Add these students: "
    "Jake Alvarez grade 11, Maria Chen grade 9, Devon Hill grade 12."
)
print(response)

[Planner] received: 'Add these students: Jake Alvarez grade 11, Maria Chen grade 9, Devon Hill grade 12.'
[Planner] plan: {'intent': 'INSERT', 'entity': 'students', 'is_batch': True, 'records': [{'first_name': 'Jake', 'last_name': 'Alvarez', 'g
[Generator] received plan: intent=INSERT, entity=students, is_batch=True
[Validator] is_valid=True, errors=[]
[Executor] received SQL: 'INSERT INTO students (first_name, last_name, grade) VALUES (%s, %s, %s)', is_batch=True
[Executor] success: rows affected: 3
3 new students have been added to your roster: Jake Alvarez (11th grade), Maria Chen (9th grade), and Devon Hill (12th grade). They're all
```

Figure 4: Adding multiple students to the database.

6.1.1 Single Insert

The demo of a single insert is shown in Figure 3. The Planner correctly determines the correct intent and tables for adding a student to the database, and the Formatter provides natural language feedback to confirm the operation was successful without using technical terms that the band director may not be familiar with.

6.1.2 Batch Insert

The demo of a batch insert is shown in Figure 4. The planner correctly identifies this request as a batch request, allowing the generator to create multiple INSERT statements at once. The Formatter responds by confirming the total number of students added, along with their names.

```
response = chat("Show me all pieces with difficulty 3.")
print(response)

[Planner] received: 'Show me all pieces with difficulty 3.'
[Planner] plan: {'intent': 'SELECT', 'entity': 'music', 'is_batch': False, 'records': [], 'filters': {'difficulty': 3}, 'updates': {}, 're
[Generator] received plan: intent=SELECT, entity=music, is_batch=False
[Generator] sql: 'SELECT m.music_id, m.title, m.composer, m.difficulty FROM music m WHERE m.difficulty = %s', params: [3]
[Validator] is_valid=True, errors=[]
[Executor] received SQL: 'SELECT m.music_id, m.title, m.composer, m.difficulty FROM music m WHERE m.difficulty = %s', is_batch=False
[Executor] success: 1 row(s) returned
Here's the one piece in your inventory rated at difficulty level 3:

**Commando March** — composed by Karl King

That's the only piece currently cataloged at that difficulty level. If you were expecting more, it's possible other pieces haven't had the
```

Figure 5: Filtering the music library by difficulty.

```
response = chat("Add Alex Turner, an 8th grader, to the student list.")
print(response)

[Planner] received: 'Add Alex Turner, an 8th grader, to the student list.'
[Planner] plan: {'intent': 'INSERT', 'entity': 'students', 'is_batch': False, 'records': [{'first_name': 'Alex', 'last_name': 'Turner', 'g
[Generator] received plan: intent=INSERT, entity=students, is_batch=False
[Validator] is_valid=False, errors=['Record: grade must be an integer between 9 and 12 (got 8).']
It looks like Alex Turner's grade level isn't valid for this system, which only tracks students in grades 9–12 (high school). Please doubl
```

Figure 6: Validation layer enforces business rules.

6.1.3 Select By Difficulty

The demo of a query operation is shown in Figure 5. The prompt uses music terminology like "piece," which means musical composition, instead of terminology hard-coded into the database like "music." Band directors commonly use both words to define the compositions their band plays, so it is important that the Planner can determine the intent and tables through music terminology. In addition, the Generator produces the correct SQL statement based on the plan.

6.1.4 Validation Check

The demo of validation enforcement is shown in Figure 6. In this instance, the operation is unsuccessful, not because the demo failed, but because the Validator correctly caught a business rules error. The Planner correctly determined the intent, and the Generator built the proper SQL statement for the intent. When the plan was sent to the Validator, it found that the value for *grade* (8) was outside of the acceptable range (between 9 and 12), bypassing the Executor and passing the error to Formatter.

6.2 Limitations & Expansion

The demo was successful in showcasing the system's functionality, but there are some limitations that can be addressed and expansions to improve the system in the future. First, because the Planner is LLM-based, ambiguous phrasing can produce unexpected plans. The Planner system prompts were not always accurate and required several iterations to pass the tests. The current prompts are improved, but more testing can be done to catch even more ambiguous cases.

In addition, the demo for SELECT operations is not incredibly complex. It is great for demonstrating its use, but more complex queries can be tested since it is the only operation that is not generated deterministically. Finally, batch inserts are currently limited to one table at a time. A useful expansion could include modifying the system prompt to support batches of multiple tables at once.

Additional expansions to the system can also address unique band director logistics. For example, adding inventory to a database is common among administrative tasks, but band directors specifically deal with inventories of music PDFs. A useful expansion would be a component that can parse name, composer, instrumentation, difficulty, and more from the music PDFs and send those to the Planner directly, rather than the band director typing out the music information.

References

[1] Chris McKie. Band class agent. https://github.com/chrismckie/Band-Class-Agent. GitHub.